

# Guida pratica NetworkX per prove TdP

*Metodi utili per costruzione grafo, statistiche, componenti e cammini*

Questa guida è pensata per le tracce d'esame in cui si costruisce un grafo con NetworkX e poi si chiedono statistiche come numero di nodi/archi, archi di peso maggiore, componenti connesse, gradi, pesi entranti/uscenti e cammini. L'obiettivo è sapere rapidamente quale metodo usare, cosa restituisce e su quale tipo di grafo funziona.

## 1. Scelta del tipo di grafo

| Traccia dice...                | Usa                           | Nota pratica                                                                                                          |
|--------------------------------|-------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| grafo semplice / non orientato | <code>nx.Graph()</code>       | Un arco A-B è uguale a B-A. Se aggiungi entrambi, resta un solo arco e il peso viene sovrascritto se non lo gestisci. |
| grafo orientato / diretto      | <code>nx.DiGraph()</code>     | A->B e B->A sono due archi diversi. Utile quando la traccia parla di verso, entrante, uscente.                        |
| grafo pesato                   | attributo <code>weight</code> | Il peso non cambia il tipo di grafo: è un attributo dell'arco, di solito <code>data={"weight": valore}</code> .       |

## 2. Metodi base di costruzione e lettura

### Creare, svuotare, aggiungere nodi e archi

| Metodo                                  | Restituisce                  | Grafo         | Spiegazione pratica                                                                                                                |
|-----------------------------------------|------------------------------|---------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>nx.Graph()</code>                 | oggetto <code>Graph</code>   | Non orientato | Crea un grafo semplice non diretto. Da usare se la traccia dice "grafo semplice", "non orientato", "componenti connesse".          |
| <code>nx.DiGraph()</code>               | oggetto <code>DiGraph</code> | Orientato     | Crea un grafo diretto. Da usare se la traccia parla di verso degli archi, archi uscenti/entranti, cammini che rispettano il verso. |
| <code>G.clear()</code>                  | <code>None</code>            | Entrambi      | Svuota completamente il grafo. Utile prima di ricostruirlo quando l'utente cambia selezioni.                                       |
| <code>G.add_node(n)</code>              | <code>None</code>            | Entrambi      | Aggiunge un singolo nodo. Il nodo può essere un id, una stringa o un oggetto <code>dataclass</code> hashable.                      |
| <code>G.add_nodes_from(lista)</code>    | <code>None</code>            | Entrambi      | Aggiunge tanti nodi insieme. È il metodo più comodo quando i nodi arrivano da una query DAO.                                       |
| <code>G.add_edge(u, v)</code>           | <code>None</code>            | Entrambi      | Aggiunge un arco. Nel <code>Graph</code> è non orientato; nel <code>DiGraph</code> crea <code>u-&gt;v</code> .                     |
| <code>G.add_edge(u, v, weight=p)</code> | <code>None</code>            | Entrambi      | Aggiunge un arco pesato. Poi il peso si legge con <code>data["weight"]</code> .                                                    |
| <code>G.has_edge(u, v)</code>           | <code>bool</code>            | Entrambi      | Controlla se esiste l'arco. Nel <code>DiGraph</code> controlla proprio <code>u-&gt;v</code> , non <code>v-&gt;u</code> .           |

### Statistiche immediate del grafo

| Metodo                             | Restituisce                                | Grafo    | Spiegazione pratica                                                                                           |
|------------------------------------|--------------------------------------------|----------|---------------------------------------------------------------------------------------------------------------|
| <code>G.number_of_nodes()</code>   | <code>int</code>                           | Entrambi | Numero di nodi/vertici presenti nel grafo.                                                                    |
| <code>G.number_of_edges()</code>   | <code>int</code>                           | Entrambi | Numero di archi. In un <code>DiGraph</code> A->B e B->A contano come 2 archi.                                 |
| <code>G.nodes()</code>             | <code>NodeView</code>                      | Entrambi | Vista dei nodi. Puoi ciclarla con <code>for n in G.nodes()</code> .                                           |
| <code>list(G.nodes())</code>       | <code>list</code>                          | Entrambi | Lista vera dei nodi. Utile per dropdown o ordinamenti.                                                        |
| <code>G.edges()</code>             | <code>EdgeView</code>                      | Entrambi | Vista degli archi senza dati. Ogni elemento è una coppia (u, v).                                              |
| <code>G.edges(data=True)</code>    | <code>EdgeDataView</code>                  | Entrambi | Vista degli archi con dizionario attributi. Ogni elemento è (u, v, data). Per pesi serve quasi sempre questo. |
| <code>G.get_edge_data(u, v)</code> | <code>dict</code> oppure <code>None</code> | Entrambi | Restituisce il dizionario dati dell'arco, ad esempio                                                          |

|  |  |  |                                                       |
|--|--|--|-------------------------------------------------------|
|  |  |  | {"weight": 7}. Se l'arco non esiste restituisce None. |
|--|--|--|-------------------------------------------------------|

### 3. Grado, archi incidenti, pesi entranti e uscenti

#### Gradi e archi di un nodo

| Metodo                       | Restituisce           | Grafo                                | Spiegazione pratica                                                                                           |
|------------------------------|-----------------------|--------------------------------------|---------------------------------------------------------------------------------------------------------------|
| G.degree[n]                  | int                   | Entrambi                             | Numero di archi incidenti al nodo. In un DiGraph somma entranti + uscenti.                                    |
| G.degree(n)                  | int                   | Entrambi                             | Stessa idea: grado del nodo n. Comodo negli ordinamenti.                                                      |
| G.degree(n, weight="weight") | numero                | Entrambi                             | Somma dei pesi degli archi incidenti al nodo. In DiGraph somma entranti + uscenti.                            |
| G.edges(n, data=True)        | archi incidenti       | Graph; anche DiGraph ma solo uscenti | Nel Graph dà gli archi collegati a n. Nel DiGraph, attenzione: di default restituisce gli archi uscenti da n. |
| G.neighbors(n)               | iteratore nodi vicini | Entrambi                             | Nel Graph sono i vicini. Nel DiGraph equivale ai successori, cioè nodi raggiunti da n.                        |
| G[n]                         | dict-like adjacency   | Entrambi                             | Accesso rapido ai vicini e ai dati degli archi: G[u][v]["weight"].                                            |

#### Solo per grafi orientati

| Metodo                           | Restituisce          | Grafo     | Spiegazione pratica                                                   |
|----------------------------------|----------------------|-----------|-----------------------------------------------------------------------|
| G.out_degree[n]                  | int                  | Orientato | Numero di archi uscenti dal nodo n.                                   |
| G.in_degree[n]                   | int                  | Orientato | Numero di archi entranti nel nodo n.                                  |
| G.out_degree(n, weight="weight") | numero               | Orientato | Somma dei pesi degli archi uscenti. Utile per "peso archi uscenti".   |
| G.in_degree(n, weight="weight")  | numero               | Orientato | Somma dei pesi degli archi entranti. Utile per "peso archi entranti". |
| G.out_edges(n, data=True)        | lista/vista di archi | Orientato | Archi uscenti da n: (n, successore, data).                            |
| G.in_edges(n, data=True)         | lista/vista di archi | Orientato | Archi entranti in n: (predecessore, n, data).                         |
| G.successors(n)                  | iteratore nodi       | Orientato | Nodi raggiungibili direttamente da n con un arco uscente.             |
| G.predecessors(n)                | iteratore nodi       | Orientato | Nodi che arrivano direttamente a n con un arco entrante.              |

#### Esempi pratici su gradi e pesi

```
# 5 nodi con più archi uscenti, poi somma pesi uscenti
nodi = sorted(G.nodes(), key=lambda n: G.out_degree[n], reverse=True)[:5]
for n in nodi:
    peso_uscente = G.out_degree(n, weight="weight")
    print(n, G.out_degree[n], peso_uscente)

# influenza = somma pesi uscenti - somma pesi entranti
influenza = {}
for n in G.nodes():
    out_w = G.out_degree(n, weight="weight")
    in_w = G.in_degree(n, weight="weight")
    influenza[n] = out_w - in_w

# nodi ordinati per influenza decrescente
ordinati = sorted(G.nodes(), key=lambda n: influenza[n], reverse=True)

# Peso massimo/minimo degli archi incidenti a un nodo in un Graph
# Utile per: "ordina i nodi della componente per peso massimo/minimo degli archi incidenti"
def peso_max_incidente(G, n):
    pesi = [data["weight"] for _, _, data in G.edges(n, data=True)]
```

```

return max(pesi) if pesi else 0

def peso_min_incidente(G, n):
    pesi = [data["weight"] for _, _, data in G.edges(n, data=True)]
    return min(pesi) if pesi else 0

nodi_ordinati = sorted(comp, key=lambda n: peso_max_incidente(G, n), reverse=True)

```

## Ordinare archi e leggere i pesi

| Metodo                               | Restituisce | Grafo    | Spiegazione pratica                                                                          |
|--------------------------------------|-------------|----------|----------------------------------------------------------------------------------------------|
| for u, v, data in G.edges(data=True) | tuple       | Entrambi | Cicla tutti gli archi con i dati. data è un dizionario, quindi data["weight"] legge il peso. |
| sorted(G.edges(data=True), key=...)  | list        | Entrambi | Ordina gli archi secondo una funzione, per esempio il peso.                                  |
| data.get("weight", 0)                | numero      | Entrambi | Lettura sicura del peso: se manca weight restituisce 0.                                      |

```

# 5 archi di peso maggiore
archi_top = sorted(
    G.edges(data=True),
    key=lambda e: e[2].get("weight", 0),
    reverse=True
)[:5]

for u, v, data in archi_top:
    print(u, v, data["weight"])

```

## 4. Componenti connesse

### Componenti per grafi non orientati

| Metodo                            | Restituisce       | Grafo              | Spiegazione pratica                                                                             |
|-----------------------------------|-------------------|--------------------|-------------------------------------------------------------------------------------------------|
| nx.connected_components(G)        | generatore di set | Solo non orientato | Restituisce le componenti connesse. Ogni componente è un set di nodi. Non funziona sui DiGraph. |
| nx.number_connected_components(G) | int               | Solo non orientato | Numero di componenti connesse.                                                                  |
| nx.node_connected_component(G, n) | set               | Solo non orientato | Componente connessa che contiene il nodo n.                                                     |
| G.subgraph(comp)                  | vista SubGraph    | Entrambi           | Crea il sottografo indotto dai nodi in comp. Se vuoi modificarlo usa .copy().                   |
| G.subgraph(comp).copy()           | nuovo grafo       | Entrambi           | Copia indipendente del sottografo. Utile per lavorare sulla componente maggiore.                |

```

# Numero componenti e componente maggiore in un Graph
componenti = list(nx.connected_components(G))
print(len(componenti))

comp_max = max(componenti, key=len)

# Nodi della componente maggiore ordinati per grado decrescente
nodi_ordinati = sorted(comp_max, key=lambda n: G.degree[n], reverse=True)
for n in nodi_ordinati:
    print(n, G.degree[n])

```

## Componenti per grafi orientati

| Metodo                                                  | Restituisce       | Grafo                      | Spiegazione pratica                                                                                                                                   |
|---------------------------------------------------------|-------------------|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>nx.weakly_connected_components(G)</code>          | generatore di set | Solo orientato             | Componenti debolmente connesse: ignora il verso degli archi. È quella richiesta spesso quando la traccia dice "debolmente connesse".                  |
| <code>nx.number_weakly_connected_components(G)</code>   | int               | Solo orientato             | Numero di componenti debolmente connesse.                                                                                                             |
| <code>nx.strongly_connected_components(G)</code>        | generatore di set | Solo orientato             | Componenti fortemente connesse: da ogni nodo della componente si può raggiungere ogni altro rispettando i versi.                                      |
| <code>nx.number_strongly_connected_components(G)</code> | int               | Solo orientato             | Numero di componenti fortemente connesse.                                                                                                             |
| <code>G.to_undirected()</code>                          | Graph             | Orientato -> non orientato | Converte ignorando il verso. Può essere utile, ma nelle tracce è meglio usare <code>weakly_connected_components</code> se chiedono componenti deboli. |

```
# Componenti debolmente connesse in un DiGraph
componenti = list(nx.weakly_connected_components(G))
print(len(componenti))

comp_max = max(componenti, key=len)
for n in comp_max:
    print(n)
```

## 5. Cammini e percorsi

### Metodi NetworkX per cammini

| Metodo                                                            | Restituisce         | Grafo                 | Spiegazione pratica                                                                                                              |
|-------------------------------------------------------------------|---------------------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <code>nx.shortest_path(G, source, target)</code>                  | list di nodi        | Entrambi              | Trova un cammino minimo. Utile solo se la traccia chiede il più corto; per "percorso massimo" di solito serve ricorsione custom. |
| <code>nx.shortest_path(G, source, target, weight="weight")</code> | list di nodi        | Entrambi              | Cammino a peso minimo usando i pesi. Non massimizza il peso.                                                                     |
| <code>nx.all_simple_paths(G, source, target, cutoff=None)</code>  | generatore di liste | Entrambi              | Genera tutti i cammini semplici tra source e target. Attenzione: può esplodere su grafi grandi.                                  |
| <code>nx.has_path(G, source, target)</code>                       | bool                | Entrambi              | Dice se esiste almeno un cammino tra due nodi. Nei DiGraph rispetta la direzione.                                                |
| <code>nx.descendants(G, source)</code>                            | set                 | Orientato soprattutto | Nodi raggiungibili da source seguendo i versi degli archi.                                                                       |
| <code>nx.ancestors(G, source)</code>                              | set                 | Orientato soprattutto | Nodi da cui si può raggiungere source seguendo i versi.                                                                          |
| <code>nx.is_directed_acyclic_graph(G)</code>                      | bool                | Orientato             | Dice se il grafo orientato è un DAG. Utile solo se vuoi applicare metodi da DAG.                                                 |
| <code>nx.topological_sort(G)</code>                               | generatore di nodi  | DAG orientato         | Ordina i nodi in modo compatibile con gli archi. Funziona solo se non ci sono cicli.                                             |

Nelle tracce TdP il "percorso massimo" quasi sempre ha vincoli particolari: peso crescente/decrescente, età crescente, durata crescente, lunghezza fissata, massimo punteggio. In questi casi NetworkX non basta da solo: si usano soprattutto `G.successors(n)`, `G.neighbors(n)`, `G.out_edges(n, data=True)` dentro una ricorsione DFS.

```
# Schema base DFS per cammino semplice in un DiGraph
# Personalizza la condizione in base alla traccia: peso crescente, età crescente, durata crescente, ecc.
def cerca(cammino):
    ultimo = cammino[-1]

    # aggiorna soluzione migliore qui
```

```

for succ in G.successors(ultimo):      # per Graph usa G.neighbors(ultimo)
    if succ not in cammino:           # cammino semplice: niente nodi ripetuti
        # controlla qui i vincoli della traccia
        cammino.append(succ)
        cerca(cammino)
        cammino.pop()

cerca([start])

# DFS con archi e pesi in un DiGraph
def cerca(cammino, peso_precedente):
    ultimo = cammino[-1]
    for _, succ, data in G.out_edges(ultimo, data=True):
        peso = data["weight"]
        if succ not in cammino and peso < peso_precedente: # esempio: pesi decrescenti
            cammino.append(succ)
            cerca(cammino, peso)
            cammino.pop()

```

## 6. Ricette già pronte per le statistiche più frequenti

| Richiesta della traccia                           | Grafo    | Metodo/idea                                                                          |
|---------------------------------------------------|----------|--------------------------------------------------------------------------------------|
| Numero nodi e archi                               | Entrambi | G.number_of_nodes() e G.number_of_edges().                                           |
| Tre/cinque archi di peso maggiore                 | Entrambi | sorted(G.edges(data=True), key=lambda e: e[2]["weight"], reverse=True)[:k].          |
| Numero componenti connesse                        | Graph    | len(list(nx.connected_components(G))) oppure nx.number_connected_components(G).      |
| Componente connessa maggiore                      | Graph    | max(nx.connected_components(G), key=len).                                            |
| Nodi della componente maggiore ordinati per grado | Graph    | sorted(comp, key=lambda n: G.degree[n], reverse=True).                               |
| Componenti debolmente connesse                    | DiGraph  | list(nx.weakly_connected_components(G)).                                             |
| Nodi con più archi uscenti                        | DiGraph  | sorted(G.nodes(), key=lambda n: G.out_degree[n], reverse=True).                      |
| Somma pesi uscenti                                | DiGraph  | G.out_degree(n, weight="weight").                                                    |
| Somma pesi entranti                               | DiGraph  | G.in_degree(n, weight="weight").                                                     |
| Influenza = uscenti - entranti                    | DiGraph  | G.out_degree(n, weight="weight") - G.in_degree(n, weight="weight").                  |
| Peso massimo archi incidenti                      | Graph    | max(data["weight"] for _, data in G.edges(n, data=True)), con gestione nodo isolato. |
| Peso minimo archi incidenti                       | Graph    | min(data["weight"] for _, data in G.edges(n, data=True)), con gestione nodo isolato. |
| Cammino rispettando il verso                      | DiGraph  | Usa G.successors(n) oppure G.out_edges(n, data=True).                                |
| Cammino in grafo non orientato                    | Graph    | Usa G.neighbors(n) oppure G.edges(n, data=True).                                     |

## 7. Errori tipici da evitare

- Usare nx.connected\_components(G) su un DiGraph: dà errore. Per grafi orientati usa weakly\_connected\_components o strongly\_connected\_components.
- Usare weakly\_connected\_components su un Graph: è solo per grafi orientati.
- Pensare che G.degree in un DiGraph sia solo uscente: in realtà conta entranti + uscenti. Per il verso usa in\_degree/out\_degree.

- Dimenticare `data=True` quando servono i pesi degli archi.
- In un `DiGraph`, `G.neighbors(n)` equivale ai successori: non restituisce i predecessori.
- Nel `Graph`, aggiungere sia A-B sia B-A non crea due archi: è sempre lo stesso arco.
- Se un nodo è isolato, `max/min` sugli archi incidenti dà errore perché la lista è vuota: gestiscilo con valore 0 o -1 secondo la traccia.
- Per percorsi massimi con vincoli da traccia, non cercare un metodo magico `NetworkX`: usa una DFS ricorsiva personalizzata.

## 8. Mini-template da copiare nel Model

```
import networkx as nx

class Model:
    def __init__(self):
        self._graph = nx.Graph()      # oppure nx.DiGraph()
        self._idMap = {}

    def get_num_nodi(self):
        return self._graph.number_of_nodes()

    def get_num_archi(self):
        return self._graph.number_of_edges()

    def get_archi_top_peso(self, k=5):
        return sorted(
            self._graph.edges(data=True),
            key=lambda e: e[2].get("weight", 0),
            reverse=True
        )[:k]

    def get_componenti_graph(self):
        return list(nx.connected_components(self._graph))

    def get_componente_maggiore_graph(self):
        comps = list(nx.connected_components(self._graph))
        if not comps:
            return []
        comp = max(comps, key=len)
        return sorted(comp, key=lambda n: self._graph.degree[n], reverse=True)

    def get_componenti_deboli_digraph(self):
        return list(nx.weakly_connected_components(self._graph))
```

**Nota sulle fonti:** i nomi e i comportamenti dei metodi sono coerenti con la documentazione stabile `NetworkX 3.6.1`. Per le prove TdP, le parti più importanti sono `componenti`, `degree/in_degree/out_degree`, `edges(data=True)`, `successors/predecessors` e `DFS custom`.